

Extending TRIE for Finding Invariants in Rust Crates

Sidney Richardson
sidney.richardson@yale.edu

Advisors:
Lin Zhong
lin.zhong@yale.edu
Ramla Ijaz
ramla.ijaz@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Science in Computer Science*

Department of Computer Science
Yale University
Dec 11, 2025

Contents

1	Introduction	4
2	Background	6
2.0.1	Rust’s Type System	6
2.0.2	The Typestate Pattern	6
2.0.3	LLMs and Code Representation	8
3	Code Design and Implementation	9
4	Methodology and Evaluation	12
5	Results	14
6	Future Work	17
7	Conclusions	18
8	References	19

Extending TRIE for Finding Invariants in Rust Crates

Sidney Richardson

Abstract

Large language models often struggle to preserve Rust’s implicit invariants when refactoring or transforming code, particularly those tied to ownership, lifetimes, and type-state behavior. These invariants are rarely written down explicitly; instead, they emerge from patterns involving generics, trait bounds, or ownership rules. As a result, automated systems frequently overlook constraints that are essential to maintaining program correctness. This project develops an initial three-stage pipeline that retrieves relevant code snippets, identifies potential state transitions and invariants, and validates them through a structured prompting and de-hallucination process.

The pipeline begins with code summarization and selection, which uses keyword-based semantic search to locate snippets likely to encode typestate behavior. Next, an invariant-detection stage analyzes each snippet and constructs preliminary typestate entries capturing states, transitions, and supporting evidence. A final validation stage evaluates these entries for grounding, removes unsupported inferences, corrects entity names, and enforces that transitions align with actual function signatures.

Applied to small Rust crates such as `OnceCell`, the system successfully produces typestate tables with explicit, line-level evidence, revealing both correct inferences and areas where LLMs tend to misinterpret or omit key constraints. Across iterative versions of the pipeline, improvements in retrieval, grounding, and validation yielded more consistent and interpretable results while exposing tradeoffs between recall and hallucination control.

Although still exploratory, these findings suggest that structured retrieval and validation can help LLMs recover meaningful typestate information from real Rust code. The system provides a preliminary foundation for future tools aimed at supporting invariant-aware Rust-to-Rust transformations and guiding models toward more reliable reasoning about Rust programs.

1 Introduction

Much recent work has focused on using large language models (LLMs) to translate, refactor, or improve existing Rust code. However, the equivalence between two programs depends not only on syntactic correctness, but also on preserving the same set of invariants, conditions that must always hold true during execution, between the two programs.

Many of these invariants have to do with two things: temporal order and state consistency. Invariants involving temporal order ensure that operations occur in a legal sequence. Invariants for state consistency ensure that resources are used only in states where operations are valid. In Rust, unlike other languages, these invariants can be guaranteed with the type state pattern, a design pattern that encodes information about an object's run-time state in its compile time state [1]. The unique ability for Rust to enforce temporal order and state consistency is due to its affine type system, which enforces single ownership of resources and prevents data races and memory corruption [2].

However, these invariants are often implicit, appearing through generics, trait bounds, ownership rules, or simply through comments and informal code structure rather than explicit specifications. This makes them difficult for automated systems or AI models to detect and reason about directly. This project seeks to extend the TRIE (Textual Retrieval and Invariant Extraction) framework to Rust codebases in order to automatically identify and extract these invariants that arise through the typestate pattern. The system combines code navigation, code parsing, and language-model reasoning to discover and represent type state transitions that govern a Rust crate. By doing so, it enables an AI agent to traverse structured representations of code, infer ownership and state-transition relationships.

By making these invariants explicit, the system bridges the gap between Rust's compiler-level guarantees and higher-level program understanding. A system that can automatically uncover these invariants can help make these guarantees more transparent, improving the interpretability of Rust programs and enhancing the ability of AI systems to reason about correctness beyond surface-level syntax. In the long term, this approach could assist in the development of tools that verify or rewrite code while preserving its underlying safety properties.

So far, the project has made solid progress toward this goal. The system now includes a working three-stage pipeline that retrieves relevant Rust code, generates initial typestate entries, and applies a validation pass to remove unsupported or incorrect results. A structured format for representing states, transitions, and evidence has been developed, and the system can produce typestate tables on small crates like `OnceCell`. The prompting structure, storage of intermediate outputs, and benchmarking setup have been refined across several iterations, helping identify issues such as duplication, grounding errors, and incomplete transitions. While the system is still evolving and not yet fully reliable,

these components establish a functional starting point for automated analysis of implicit typestate behavior in Rust code.

2 Background

2.0.1 Rust's Type System

Rust's type system is designed to enforce safety and correctness at compile time by strictly controlling how data is used and shared. Unlike languages that depend on runtime checks or garbage collection, Rust achieves memory safety through an *affine type system*, which allows each value to be used at most once unless it explicitly implements a duplication trait such as Copy or Clone [2]. This restriction ensures that ownership of resources is always well defined and that no two variables can simultaneously claim responsibility for freeing or mutating the same data. Rust's ownership model builds on this foundation through three key concepts: ownership, borrowing, and lifetimes. Ownership determines which variable is responsible for releasing a resource when it goes out of scope. Borrowing allows temporary access to owned data, which can be either immutable and shared or mutable and exclusive, but never both at the same time. Lifetimes define how long a reference remains valid and ensure that no reference outlives the data it points to. Together, these rules form a system of compile-time guarantees that prevent common sources of error such as double frees, dangling pointers, and data races [3]. Additionally, by enforcing how resources are created, borrowed, and released, Rust's type system not only guarantees memory safety but also enables the expression of higher-level program invariants through the *typestate pattern*.

2.0.2 The Typestate Pattern

The typestate pattern is a design pattern in which information about an object's runtime state is encoded in its compile time type. Three key properties are key to the type state system in Rust:

1. State-specific operations: Certain methods or functions are only available when an object is in a specific state.
2. Type-level state encoding: The object's type encodes its state, ensuring that attempts to perform an operation in an invalid state fail to compile.
3. State transitions: Operations can consume a value in one state and return a new value in another, changing the type-level state in addition to the runtime state. This guarantees that operations from the previous state are no longer accessible once a transition has occurred.

These three properties are evident in the following piece of code:

```

enum Closed {}
enum Open {}

trait State {}
impl State for Closed {}
impl State for Open {}

struct File<S: State> {
    name: String,
    marker: std::marker::PhantomData<S>,
}

impl File<Closed> {
    fn new(name: &str) -> Self {
        File {
            name: name.to_string(),
            marker: std::marker::PhantomData,
        }
    }

    fn open(self) -> File<Open> {
        println!("Opening file: {}", self.name);
        File {
            name: self.name,
            marker: std::marker::PhantomData,
        }
    }
}

impl File<Open> {
    fn read(&self) {
        println!("Reading contents of: {}", self.name);
    }

    fn close(self) -> File<Closed> {
        println!("Closing file: {}", self.name);
        File {
            name: self.name,
            marker: std::marker::PhantomData,
        }
    }
}

```

Figure 1

The example above demonstrates the `typestate` pattern by encoding the file’s possible runtime states—`Closed` and `Open`—directly into its compile-time type. Each state becomes a distinct type parameter, which restricts which methods are available at any given moment. For instance, the `read` method is only implemented for `File!Open`, so attempting to read from a closed file is rejected by the compiler before the program can run. Likewise, the `open` and `close` methods consume `self` and return a new `File` value whose type reflects the new state. This consumption ensures that once a file transitions from `Closed` to `Open`, the old `Closed` value no longer exists and cannot be used accidentally. The compiler therefore enforces both temporal ordering—operations must occur in the correct sequence—and state consistency—each operation is only valid in the state it is defined for. In this way, the type system replaces what would normally be runtime checks with static guarantees, making illegal sequences of operations unrepresentable in the program itself.

2.0.3 LLMs and Code Representation

LLMs can handle many Rust-related tasks, but they still process programs primarily as linear text, which limits their ability to capture the structural relationships that govern correct behavior. Because an LLM can only see a small portion of a codebase at once, important information spread across files or distant parts of a function may be missed. This becomes especially relevant in Rust, where correctness often depends on implicit constraints involving ownership, lifetimes, control-flow ordering, and `typestate` behavior. Studies show that LLMs tend to recognize common or familiar code patterns but struggle with more complex dependencies, uncommon state transitions, and invariants that require reasoning beyond the visible snippet [4][5].

For Rust-to-Rust refactoring, these limitations matter: a transformation may look syntactically correct but silently break an invariant tied to initialization order, borrowing rules, or valid state transitions. This project aims to reduce that risk by making those implicit relationships explicit. By retrieving the specific snippets where state changes occur, identifying the states and transitions involved, and attaching concrete evidence lines, the system gives the model more structured guidance about the constraints that must be preserved. This additional context does not fix the fundamental limitations of LLMs, but it helps keep transformations grounded in the code’s actual semantics rather than in patterns the model assumes.

3 Code Design and Implementation

This autonomous system was built using Python and OpenAI’s API, specifically GPT-4o. It functions as a command-line tool that takes a Rust crate as input and attempts to automatically identify the typestate-style invariants embedded within it. The goals of the project can be summarized as follows. The system should be effective, meaning it can successfully detect invariants. It should be adaptable, capable of running on a variety of Rust repositories without requiring manual tuning. It should also be efficient, minimizing unnecessary API calls and redundant analysis steps. Finally, it should be transparent, providing clear evidence lines that show exactly where each invariant was found in the source code. The system can be understood as a three-stage process.

Code Summarization and Selection is where the workflow begins. Here, the agent performs a semantic search over the Rust crate, looking for code patterns that are likely to encode typestate behavior, such as the use of phantom data, enum-based state markers, or functions that consume and return different generic types. In practice, this step amounted to a keyword-driven search using terms associated with state changes and initialization patterns. The output is a list of code blocks that match these surface-level indicators and are therefore likely to contain explicit or implicit typestate structure.

Invariant Detection examines each returned code block to determine whether it contains a state transition or an implicit invariant. If it does, the system produces an entry in a typestate table, describing the entity, its states, the inferred invariants, the state transitions, and the evidence lines. The entity refers to the struct or type being analyzed; the state is a compile-time marker that reflects some meaningful runtime behavior; the invariants capture constraints that always hold in that state; and the transitions represent functions that move the entity between states. These states and transitions may be either explicit ones already present in the crate or implicit ones inferred from how the code behaves. Finally, the evidence lines identify the specific portions of the snippet supporting each inference.

Validation introduces a de-hallucination step that evaluates the entries produced in the prior stage. This component checks whether each proposed invariant is sufficiently grounded in the code snippets, removes unsupported or hallucinated entries, corrects minor issues such as incorrect entity names, and ensures that transitions align with actual function signatures. By serving as a post-processing filter, it stabilizes the system’s output and improves both correctness and consistency.

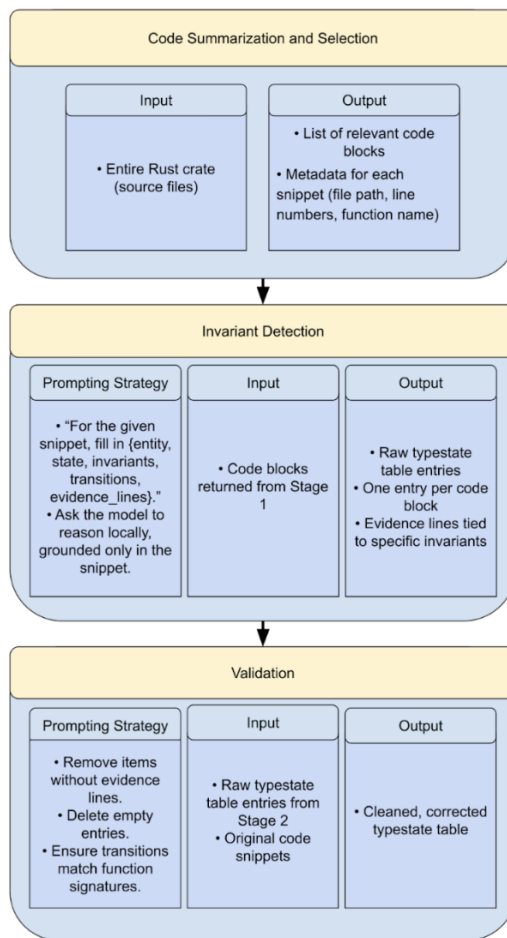


Figure 2

Code Navigation

This project used Octocode’s embedding and search tools to retrieve the Rust code most relevant to typestate analysis. Embeddings were generated using the configured OpenAI provider, with both code and text embeddings produced by the `openai:text-embedding-3-large` model. In this setup, the codebase is effectively split into linear chunks and converted into embeddings without considering how those snippets depend on or relate to one another. As a result, the retrieval step reflects semantic similarity at the chunk level rather than any deeper structural or dependency-aware understanding of the crate. These embeddings powered the system’s similarity-based search over the indexed repository, returning code blocks that matched the semantic intent of each query. The search system supports multiple modes, multi-query retrieval, and basic deduplication, helping the agent avoid redundant snippets and focus on the parts of the crate most likely to encode state transitions or invariants. Octocode’s combined embedding and search pipeline provided an efficient and practical way to navigate Rust codebases. Recent research has proposed using LLM agents to construct and execute code-structure-aware queries over a graph representation of a codebase. These systems let the model navigate a repository by issuing graph queries rather than relying only on similarity search, which can make retrieval more precise, more consistent, and more aligned with the actual relationships encoded in the code[4]. Motivated by these results, I experimented with incorporating a similar query-generation approach into my system. In practice, however, the tool I used struggled to produce stable or fully consistent queries, and it often failed to retrieve the complete context needed for analysis. Even though this prevented the method from serving as a reliable retrieval backbone, the attempt was still useful: some of the intermediate material produced during these experiments later helped enrich the context provided to the LLM for each code snippet, improving the system’s overall reasoning despite the limitations of the approach.

4 Methodology and Evaluation

To evaluate the system, I applied the full three-stage pipeline to the `OnceCell` crate, a small Rust library that provides a value that can be written only once and then read many times. Even though the implementation uses atomic flags and an internal `Option<T>`, the external behavior reduces to two intuitive states: `Empty` and `Initialized`. In the `Empty` state, the cell holds no value, writes are still allowed, and any attempt to read the value must fail. In the `Initialized` state, the cell permanently holds a value, all read operations must succeed, and all further writes must fail. Additional invariants follow from these two states: initialization may run at most once, concurrent attempts block or return immediately, and consuming operations like `into_inner` can only succeed when the cell already holds a value. These simple but strict rules make `OnceCell` an ideal example of a type whose behavior is governed by implicit invariants rather than explicit state markers.

Methodology proceeded by running the system on the entire `OnceCell` source directory and examining how each stage transformed the input. **Code Summarization and Selection:** Code Summarization and Selection took the full crate as input and returned code blocks containing functions like `set`, `get`, and `get_mut`, as well as patterns such as `Option`-wrapped fields that signal state transitions. The output of this stage was a list of snippets with file paths and line numbers, representing the program locations most likely to encode state behavior. An example output can be seen in Figure 3.

Invariant Detection: Invariant Detection examined each selected snippet and generated preliminary typestate entries based on the behaviors visible in the code. For example, from the code shown in Figure 3 (corresponding to the lines 138–167 returned by the system), the model inferred the internal state machine used during initialization. By observing how `initialize_inner` compares and updates the `AtomicU8` state, the system identified the states `INCOMPLETE`, `RUNNING`, and `COMPLETE`, along with transi-

```
"results": [
  {
    "result_number": 7,
    "file": "src/imp_pl.rs",
    "language": "rust",
    "lines": "138-167",
    "similarity": 0.2429,
    "content": [
      "139 | #[inline(never)]",
      "140 | fn initialize_inner(state: &AtomicU8, init: &mut dyn FnMut()
-> bool) {",
      "141 |     loop {",
      "142 |         let exchange =",
      "... (23 more lines)",
      "166 |     }",
      "167 | }",
      "168 | }"
    ]
  }, ..
]
```

Figure 3

```

{
  "file": "src/imp_pl.rs",
  "line_range": { "start": 139, "end": 167 },
  "tpestate_table": [
    {
      "entity": "StateMachine",
      "state": "INCOMPLETE",
      ...
    },
    {
      "entity": "StateMachine",
      "state": "RUNNING",
      ...
    },
    {
      "entity": "StateMachine",
      "state": "COMPLETE",
      "invariants": ["COMPLETE -> no further legal transitions"],
      "transitions": ["COMPLETE -> (none)"],
      "evidence_lines": [151, 147],
      "qualified_name": "once_cell.src.imp_pl.initialize_inner"
    }
  ]
}

```

Figure 4

tions such as INCOMPLETE $\rightarrow$ RUNNING when initialization begins and RUNNING $\rightarrow$ COMPLETE when the initializer succeeds. It also extracted invariants tied to these states, including that initialization can only proceed from INCOMPLETE, that the RUNNING state represents an active initialization attempt, and that COMPLETE is a terminal condition. Across other snippets, the system detected invariants related to whether the cell contains a value and which operations become valid in each state. The output of this stage was a set of raw tpestate entries, each listing the inferred states, their invariants, any transitions, and the specific evidence lines supporting those conclusions. An example output can be seen in Figure 4.

Validation: The model initially inferred invariants and transitions even when a snippet did not provide enough evidence to support them. For example, given a snippet containing only the definition of OnceCell and its state constants, the model hallucinated a full tpestate protocol despite the absence of any methods showing how the state actually changes. Validation caught this: because the snippet offered no direct evidence of transitions or state-specific behavior, these entries were removed. During this process, validation also corrected entity names when a qualified name was available. A qualified name is the fully scoped path to the item the snippet comes from which gives unambiguous information about which type or function the entry refers to. Validation used this to fix or disambiguate entity names. It also discarded transitions that did not match real function signatures. The result was a cleaned tpestate table containing only entries with explicit support drawn directly from the code. Together, these steps provide a systematic evaluation of how well the pipeline identifies and represents tpestate structure in a real Rust crate. By applying the workflow to OnceCell, the system demonstrates its ability to detect implicit invariants, tie them to program locations, and produce stable, interpretable outputs that reflect the crate’s true behavior.

5 Results

Across the project iterations, performance steadily improved but revealed a consistent tradeoff between recall and hallucination control. The earliest agent retrieved code reliably but struggled to extract meaningful invariants, often producing duplicate or incomplete entries. The second version strengthened semantic search, which improved recall but also increased hallucinations and led to states and transitions that were not grounded in the actual code. The third version introduced qualified names and more explicit reasoning steps, which produced far more correct invariants and fuller typestate tables, though unsupported states and repeated entries still appeared. The final version added a dedicated de-hallucination stage, removing unsupported transitions and correcting entity names while maintaining strong recall. Although some duplication persisted, this last agent produced the most stable and accurate results, showing that high-quality retrieval combined with grounding through qualified names and explicit filtering is the most reliable strategy for typestate extraction.

The metrics used to quantify each version map directly onto the system’s goals. The Total Invariants score measures effectiveness, since the core task is to recover grounded invariants actually present in the code. Hallucination and wrong-inference penalties support transparency and correctness by discouraging invented states or behaviors. Transition Completeness supports adaptability, because identifying valid transitions with the correct function names is essential when analyzing diverse Rust crates. Penalties for duplicate entries support efficiency by preventing redundant outputs, while penalties for missing or incorrect entity names support transparency by ensuring that every invariant can be traced to the correct type or function. Together, these metrics evaluate not only how much the system finds, but how reliably, accurately, and cleanly it does so.

The following table details the outcome from running four different versions on the `personal_once_cell` repository.

Figure 5

The first agent showed partial effectiveness, reliably retrieving relevant code but identifying only a small number of invariants. It provided a useful baseline for understanding how the system behaved with minimal prompting. This version used a *simple semantic search and a two-step workflow*. It avoided hallucinations and produced structurally clean outputs, but many entries were incomplete or duplicated. As a result, effectiveness remained limited and transparency was constrained, since most entries contained too little evidence to describe full typestate behavior.

The second agent improved overall recall and captured a broader set of invariants, supporting the project’s effectiveness and adaptability goals. It incorporated a *stronger semantic-search prompt* that returned more relevant snippets and reduced the number of empty outputs. At the same time, the broader retrieval occasionally led to entries that

	Total In-variants (+1 for each invariant found including hallucinations)	Hallucination (correct or incorrect) Penalty (-1 for each hallucinated invariant)	Wrong (hallucinated) Inference Penalty	Transition Completeness (+1 for valid state transition with function name)	Duplicate Entries	Missing or Incorrect Entity Names	Total Agent Score
results/2025-11-07_16-34	4	2	0	2	2	1	4
results/2025-11-13_10-41	16	1	2	2	1	1	35.5
results/2025-11-17_16-26	44	11	2	2	3	6	73.5
results/2025-11-17_19-04	32	4	0	5	2	1	72

were not fully grounded in the underlying code. These cases reflected growing complexity rather than outright failure, but they highlighted areas where transparency and evidence-checking would need reinforcement in later versions.

The third agent made noticeable gains in completeness, producing fuller typestate tables and identifying more structured transitions. This version introduced *qualified names* and *explicit reasoning steps*, which improved grounding and reduced ambiguity in entity identification. The richer outputs demonstrated progress toward effectiveness and adaptability, though occasional unsupported transitions and repeated entries showed that consistency and efficiency were still developing. Overall, this iteration represented a meaningful step forward in the system’s ability to interpret and organize typestate information.

The fourth agent built on these improvements by placing greater emphasis on grounding and stability. A *dedicated de-hallucination step* was added to filter unsupported transitions, refine entity names, and strengthen evidence requirements. This approach improved transparency and reduced incorrect entries without substantially lowering recall. While it still produced some near-duplicate outputs and occasionally filtered aggressively, the results were more consistent and aligned more closely with the project’s goals. Rather than being a definitive solution, this version demonstrated how targeted filtering and structured prompts help balance effectiveness, accuracy, and clarity.

Together, these iterations show a clear pattern: **improving recall through stronger semantic search often increases hallucination risk, and adding grounding mechanisms like qualified names and explicit de-hallucination steps significantly stabilizes outputs.** The most successful agents combined high recall with targeted filtering and entity-name correction, highlighting the importance of prompt design and multi-stage reasoning for accurate typestate-invariant extraction.

6 Future Work

Several directions remain for improving both the accuracy and robustness of the typestate-extraction system. First, the current evaluation is limited to OnceCell, a small and relatively self-contained repository. Expanding benchmarking to a wider range of Rust crates would provide a more realistic assessment of the system’s effectiveness and adaptability, especially as typestate structures grow in complexity. It would also be valuable to move beyond libraries that could define their own typestates and evaluate the pipeline on crates that *use* those libraries. This would show whether the system can detect and reconstruct typestate behavior purely from usage patterns and APIs, rather than only from the implementations that originally define them.

Second, the system’s code-navigation strategy can be strengthened. At present, the agent relies on a single semantic-search pass, which limits code coverage. Adding an additional semantic loop, or incorporating iterative retrieval where the agent chooses new search terms after an initial pass, could reveal functions or states that the first search misses. This would also allow testing whether a fixed set of typestate-related keywords is preferable, or whether it is more effective to let the model generate new search terms dynamically as it inspects the code.

A related extension involves introducing graph-based navigation. Tools that construct call graphs, module graphs, or control-flow graphs could allow the agent to trace state transitions more systematically across files. I briefly experimented with this approach, but encountered instability in the graph queries: small changes in the prompt or query structure often produced inconsistent or incomplete results. If graph search is used in future work, additional effort will be needed to stabilize these queries and develop patterns that return reliable slices of the code graph.

Another improvement involves deduplication. Even though duplicate typestate entries are manageable for small repositories like OnceCell, they will become an obstacle in larger crates where similar patterns appear across modules. A dedicated deduplication step, applied either before or after validation, would increase efficiency and reduce noise in the final typestate tables.

Finally, transparency can be improved by adding lightweight reasoning fields to each typestate entry. Brief explanations of why a particular invariant or transition was inferred would make the system’s decisions easier to interpret and evaluate, especially when grounding relies on implicit patterns such as mutability or atomic-state changes. Together, these extensions would materially strengthen the system’s scalability, coverage, and interpretability.

7 Conclusions

This project developed an initial framework for identifying and representing typestate-style invariants in Rust code using a combination of semantic search, structured prompting, and validation. While the system is still limited and requires further refinement, it demonstrates that even implicit state transitions and invariants can be surfaced in a way that is interpretable and grounded in the original code. The pipeline offers a practical foundation for improving Rust-to-Rust transformations, program understanding, and invariant-aware code generation. With further refinement, the system can offer clearer guidance to LLMs and other tools that work with Rust code, helping them avoid mistakes tied to implicit invariants.

8 References

[1] Bugaev, S. (2017, March 10). Understanding Rust: Ownership, borrowing, lifetimes. *Medium*. <https://medium.com/@bugaevc/understanding-rust-ownership-borrowing-lifetimes-ff9ee9f79a9c>

[2] Deligiannis, P., Lal, A., Mehrotra, N., & Rastogi, A. (2023). *Fixing Rust Compilation Errors using LLMs* (arXiv:2308.05177). arXiv. <https://doi.org/10.48550/arXiv.2308.05177>

[3] Ijaz, R., Boos, K., & Zhong, L. (2024). *Combining Type Checking and Formal Verification for Lightweight OS Correctness* (arXiv:2501.00248). arXiv. <https://doi.org/10.48550/arXiv.2501.00248>

[4] Le-Cong, T., Le, B., & Murray, T. (2025). *Can LLMs Reason About Program Semantics? A Comprehensive Evaluation of LLMs on Formal Specification Inference* (arXiv:2503.04779). arXiv. <https://doi.org/10.48550/arXiv.2503.04779>

[4] Liu, X., Lan, B., Hu, Z., Liu, Y., Zhang, Z., Wang, F., Shieh, M., & Zhou, W. (2024). *CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases* (arXiv:2408.03910). arXiv. <https://doi.org/10.48550/arXiv.2408.03910>

[5] *The Typestate Pattern in Rust*. (n.d.). Retrieved December 10, 2025, from [https://cliffle.com/blog/rust-typestate/\(N.d.\)](https://cliffle.com/blog/rust-typestate/(N.d.))